

# **Optimizing Cryptocurrency Trading with Machine Learning: Predictive Analytics with Limit Order Book and Sentiment Data**

Author

Diana Gorsh

*diana.gorsh@gmail.com*

*“This paper was created as part of a WorldQuant University degree program towards an MSc in Financial Engineering. This paper is reproduced with the consent and permission of WorldQuant University.*

*All rights reserved.”*

## ***Abstract***

*This capstone project, prepared by Diana Gorsh, tackles the challenge of optimizing cryptocurrency trading through the development of a predictive analytics model that leverages machine learning to facilitate generating trading signals by utilizing limit order book and the sentiment scores along with price movements. The project is set against the backdrop of the cryptocurrency market's extreme volatility and the limitations of traditional financial analysis tools in this unique domain. It aims to bridge the gap between conventional market analysis techniques and the specific needs of cryptocurrency trading by refining machine learning methodologies for more effective decision-making processes.*

*The core of the project is the construction of a sophisticated predictive model designed to make informed trading decisions by analyzing a wide array of market variables, such as historical trade data, sentiment, volume metrics, and the depth of the limit order book, with a focus on highly liquid cryptocurrencies. The model employs data from the top ten levels of the limit order book, utilizing stationary transformations of this data—specifically, order flow and order flow imbalance—to capture the nuances of market dynamics more accurately than traditional approaches.*

*To enhance prediction accuracy, the project explores ensemble and hybrid models that integrate various deep learning architectures, such as Long Short-Term Memory (LSTM) networks and Variational Autoencoders (VAEs), addressing challenges like data overfitting and model complexity. Moreover, it incorporates sentiment analysis through FinBERT, a transformer-based model pre-trained on financial texts, to assess market sentiment as a critical input feature.*

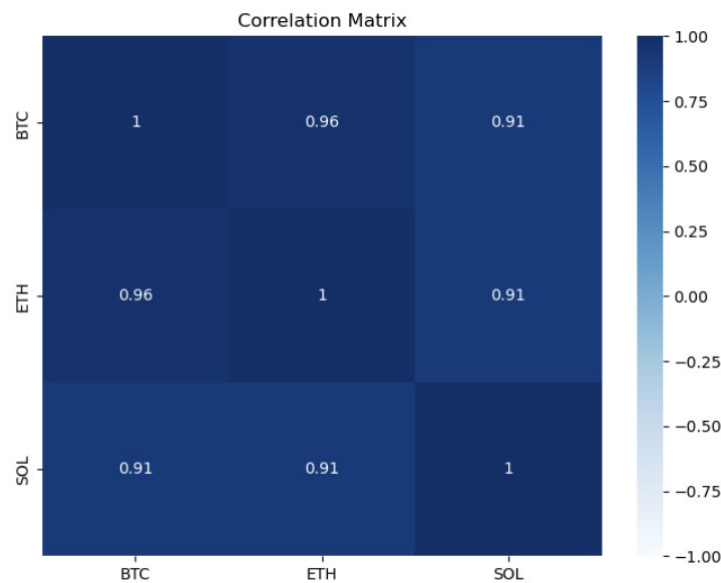
*The methodology extends beyond theoretical research, aiming to create a practical tool for cryptocurrency trading that combines market movement predictions, risk assessment, and sentiment analysis. This integrated machine learning framework endeavors to provide a comprehensive toolset for traders, allowing for more informed decisions and strategic trade execution. The project is supported by a GitHub repository and includes acknowledgments and references to foundational and recent works in the field, positioning itself as a significant contribution to both academic research and professional practice in cryptocurrency trading optimization.*

The extreme volatility in the cryptocurrency markets, coupled with the complexity of trading execution, portfolio construction and strategy optimization, contribute to challenges in making strategic investments. Traditional financial market analysis tools and techniques struggle to adapt into the cryptocurrency domain due to its unique characteristics, adding the necessity for innovative approaches in trading strategy development and portfolio management.

This capstone project aims to bridge the gap between traditional financial market analysis and the unique challenges of the cryptocurrency domain by adapting and refining machine learning methodologies to enhance decision-making processes for predicting market movements, assessing risks, and analyzing sentiment.

The central challenge addressed by this project is the development of a nuanced predictive model designed to support informed trading decisions. Such a model must interpret and analyze an extensive array of market variables including historical trade data, volume metrics, sentiment, and depth of the limit order book, focusing primarily on highly liquid cryptocurrencies such as Bitcoin (BTC), Ethereum (ETH), and Solana (SOL) against

the U.S. dollar. As we can see from the correlation matrix below (Figure 1), those 3 pairs are highly correlated.

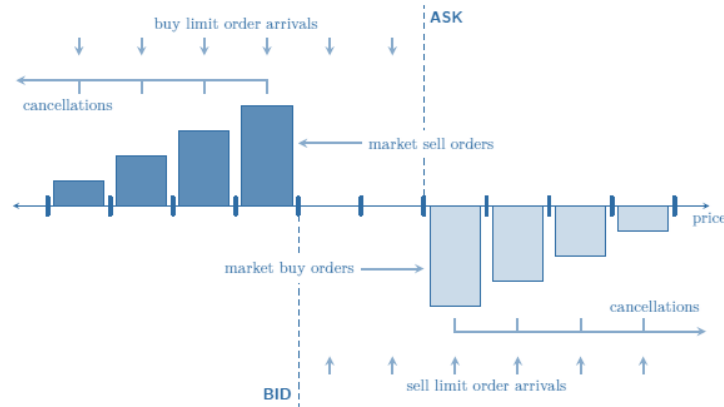


**Figure 1. Correlation Matrix for BTC, SOL and ETH**

Trading in securities today frequently utilizes a limit order book, which aggregates all buy and sell orders. This book serves to match incoming orders for execution or to be listed in the book based on their price and time of submission. The bid price represents the maximum price buyers are willing to pay, while the ask price is the minimum sellers are willing to accept. The model we are developing aims to forecast the movement of the mid-price, which is calculated as the average between the bid and ask prices.

An order's characteristics include its type (buy or sell), the quantity desired, the price at which the trade is intended, and the time it was submitted. Upon entry into the system, the exchange's matching engine attempts to pair the new order with existing ones in the book. Orders that find a match are executed as market orders, while those that don't match or only partially match become limit orders and are added to the book.

Our predictive model uses data from the top ten levels of the order book as input. Each level is specified by its price and the volume being bid or asked. On the bid side, prices decrease with each subsequent level, whereas on the ask side, prices increase. Our dataset consists of 40-variable vectors, capturing the price and volume of the top ten levels for both bids and asks, effectively providing a snapshot of the limit order book's state at each moment in time.



**Figure 2. Ask-Bid Spread**

Building on the work of Zhang et al. and others, the model is trained on stationary transformations of limit order book data - specifically order flow and order flow imbalance - to better capture the nuances of market dynamics. This approach contrasts with previous works that have typically relied on non-stationary order book states (Zhang, Zohren, and Roberts, 2020).

Stationary transformations of limit order book (LOB) data, specifically focusing on order flow and order flow imbalance, offer a refined lens through which to view and predict market dynamics. These transformations help in mitigating the challenges posed by the non-stationary nature of raw LOB data, enabling more stable and predictive modeling.

Non-stationary data, which is typical of raw LOB states, poses significant challenges for predictive modeling due to its changing mean and variance over time. This can lead to models that are less generalizable and potentially overfit to specific time periods. Stationary data, on the other hand, with its constant mean and variance, is more amenable to traditional time series analysis and forecasting techniques, leading to models that are potentially more stable and predictive over different market conditions.

The model takes as inputs representations of the first ten levels of the order book. A level is denoted by its price and volume that is bid or asked. So, as we progress down levels on the bid side of the order book, the price decreases, and as we progress down levels of the ask side, the price increases. Each observation in our dataset will be a 40-variable vector displaying the price and volume for each of the top ten bid and ask levels, giving us a truncated screenshot of the state of the limit order book at each timestep.

**Table 1. Limit Order Book (BTC-USD)**

|       | Time                | PRICE_ASK_1 | VOLUME_ASK_1 | PRICE_BID_1 | VOLUME_BID_1 | PRICE_ASK_2 | VOLUME_ASK_2 | PRICE_BID_2 | VOLUME_BID_2 | PRICE_ASK_3 |
|-------|---------------------|-------------|--------------|-------------|--------------|-------------|--------------|-------------|--------------|-------------|
| 0     | 2024-05-11 21:20:43 | 60962.91    | 0.150000     | 60962.90    | 0.015000     | 60963.01    | 0.150000     | 60961.98    | 0.049560     | 6096        |
| 1     | 2024-05-11 21:20:45 | 60962.91    | 0.035875     | 60962.90    | 0.015000     | 60963.31    | 0.023269     | 60961.98    | 0.049560     | 6096        |
| 2     | 2024-05-11 21:20:48 | 60963.99    | 0.022209     | 60962.90    | 0.015000     | 60964.01    | 0.109657     | 60961.98    | 0.049560     | 6096        |
| 3     | 2024-05-11 21:20:52 | 60961.99    | 0.150000     | 60961.98    | 0.049560     | 60963.34    | 0.012303     | 60960.79    | 0.002049     | 6096        |
| 4     | 2024-05-11 21:20:55 | 60965.19    | 0.031663     | 60961.99    | 0.015000     | 60966.63    | 0.001995     | 60961.98    | 0.049560     | 6096        |
| ...   | ...                 | ...         | ...          | ...         | ...          | ...         | ...          | ...         | ...          | ...         |
| 9996  | 2024-05-12 05:48:38 | 60975.09    | 0.441648     | 60975.08    | 0.001641     | 60976.27    | 0.011481     | 60973.66    | 0.031491     | 6097        |
| 9997  | 2024-05-12 05:48:41 | 60975.09    | 0.207801     | 60975.08    | 0.033132     | 60976.70    | 0.062866     | 60973.81    | 0.082002     | 6097        |
| 9998  | 2024-05-12 05:48:44 | 60975.09    | 0.274552     | 60975.08    | 0.031329     | 60976.71    | 0.044618     | 60973.81    | 0.082002     | 6097        |
| 9999  | 2024-05-12 05:48:47 | 60975.09    | 0.228727     | 60975.08    | 0.031329     | 60976.71    | 0.043372     | 60974.63    | 0.082001     | 6097        |
| 10000 | 2024-05-12 05:48:49 | 60975.09    | 0.223432     | 60975.08    | 0.031329     | 60977.80    | 0.020722     | 60974.63    | 0.082001     | 6097        |

10001 rows × 42 columns

We define the bid order flows (bOF) and ask order flows (aOF) at a timestamp to be 10-variable vectors computed using two consecutive order book states, where each element is given by:

$$\begin{aligned}
 \text{bOF}_{t,i} &:= \begin{cases} v_t^{i,b}, & b_t^i > b_{t-1}^i \\ v_t^{i,b} - v_{t-1}^{i,b}, & b_t^i = b_{t-1}^i \\ -v_t^{i,b}, & b_t^i < b_{t-1}^i \end{cases} \\
 \text{aOF}_{t,i} &:= \begin{cases} -v_t^{i,a}, & a_t^i > a_{t-1}^i \\ v_t^{i,a} - v_{t-1}^{i,a}, & a_t^i = a_{t-1}^i \\ v_t^{i,a}, & a_t^i < a_{t-1}^i \end{cases}
 \end{aligned} \tag{1}$$

Order Flow (OF) represents the net volume of orders entering the market. It's calculated as the difference between the volume of buy orders and sell orders within a specific timeframe and can be defined as:

$$OF_i = V_{buy,i} - V_{sell,i} \tag{2}$$

**Table 2. Bid-Ask Order Flow (BTC-USD)**

|      | bOF_1     | aOF_1     | bOF_2     | aOF_2     | bOF_3     | aOF_3     | bOF_4     | aOF_4     | bOF_5     | aOF_5     | ... | bOF_7     | aOF_7     | bOF_8     |
|------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----|-----------|-----------|-----------|
| 0    | 0.0       | -0.114125 | 0.0       | -0.023269 | -0.002049 | 0.019677  | -0.008195 | 0.016645  | -0.03914  | 0.023967  | ... | 0.036697  | 0.114994  | 0.016403  |
| 1    | 0.0       | -0.022209 | 0.0       | -0.109657 | 0.0       | -0.018496 | 0.0       | -0.025231 | -0.077092 | -0.02612  | ... | 0.0       | 0.011391  | 0.082023  |
| 2    | -0.04956  | 0.15      | -0.002049 | 0.012303  | -0.008195 | 0.02434   | -0.049214 | 0.192584  | -0.082023 | 0.018767  | ... | -0.098434 | 0.15      | -0.164056 |
| 3    | 0.015     | -0.031663 | 0.04956   | -0.001995 | 0.002049  | -0.005574 | 0.008195  | -0.034729 | 0.016403  | -0.082011 | ... | 0.011482  | -0.00034  | 0.011482  |
| 4    | 0.125     | 0.018744  | 0.015     | 0.0081    | 0.04956   | 0.034729  | 0.002049  | -0.082011 | 0.041174  | -0.011482 | ... | 0.008195  | 0.011482  | 0.030943  |
| ...  | ...       | ...       | ...       | ...       | ...       | ...       | ...       | ...       | ...       | ...       | ... | ...       | ...       | ...       |
| 9995 | -0.001641 | 0.441648  | -0.031491 | 0.011481  | -0.049202 | 0.011481  | -0.082004 | 0.050744  | -0.02     | 0.171564  | ... | -0.098409 | 0.011481  | -0.164014 |
| 9996 | 0.031491  | -0.233846 | 0.082002  | -0.062866 | -0.02     | -0.043372 | -0.000764 | -0.029291 | -0.0164   | -0.020722 | ... | -0.164014 | -0.047499 | -0.01148  |
| 9997 | -0.001803 | 0.066751  | 0.0       | -0.044618 | 0.0       | -0.029291 | 0.0       | -0.063244 | 0.0       | -0.003076 | ... | 0.0       | 0.028499  | 0.0       |
| 9998 | 0.0       | -0.045825 | 0.082001  | -0.001246 | 0.0       | 0.0       | 0.0       | 0.0       | 0.0       | 0.0       | ... | 0.0       | 0.0       | 0.0       |
| 9999 | 0.0       | -0.005295 | 0.0       | -0.020722 | 0.098405  | -0.003076 | 0.164008  | -0.0164   | 0.01148   | -0.0177   | ... | 0.01148   | -0.033898 | 0.02      |

10000 rows × 22 columns

Order Flow Imbalance (OFI) reflects the difference in the order flow between the buy and sell sides of the market, providing an indicator of which side is exerting more pressure and potentially where the price might move, and is calculated as:

$$OFI_i = (V_{buy,i} - V_{sell,i}) - (V_{buy,i-1} - V_{sell,i-1}) \quad (3)$$

where  $V_{buy,i}$  and  $V_{sell,i}$  represent the volume of buy and sell orders at time  $i$ , respectively.

**Table 3. Bid-Ask Order Flow Imbalance (BTC-USD)**

|      | OFI_1     | OFI_2     | OFI_3     | OFI_4     | OFI_5     | OFI_6     | OFI_7     | OFI_8     | OFI_9     | OFI_10    | LABEL_TICK1 | Time                |
|------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-----------|-------------|---------------------|
| 0    | 0.114125  | 0.023269  | -0.021726 | -0.024841 | -0.063107 | -0.084767 | -0.078297 | 0.013858  | -0.067977 | 0.12084   | 0.0         | 2024-05-11 21:20:45 |
| 1    | 0.022209  | 0.109657  | 0.018496  | 0.025231  | -0.050972 | -0.012885 | -0.011391 | 0.047294  | -0.003383 | 0.082013  | 0.0         | 2024-05-11 21:20:48 |
| 2    | -0.19956  | -0.014352 | -0.032536 | -0.241798 | -0.10079  | -0.166404 | -0.248434 | -0.120924 | -0.249197 | -0.04723  | 1.0         | 2024-05-11 21:20:52 |
| 3    | 0.046663  | 0.051555  | 0.007623  | 0.042925  | 0.098414  | 0.073395  | 0.011822  | 0.018572  | 0.109893  | 0.204635  | 0.0         | 2024-05-11 21:20:55 |
| 4    | 0.106256  | 0.0069    | 0.01483   | 0.08406   | 0.052657  | 0.027348  | -0.003287 | 0.030603  | 0.016403  | 0.036697  | 0.0         | 2024-05-11 21:20:58 |
| ...  | ...       | ...       | ...       | ...       | ...       | ...       | ...       | ...       | ...       | ...       | ...         | ...                 |
| 9995 | -0.443289 | -0.042971 | -0.060683 | -0.132747 | -0.191564 | -0.097836 | -0.109889 | -0.193305 | -0.098002 | -0.068353 | 0.0         | 2024-05-12 05:48:38 |
| 9996 | 0.265337  | 0.144868  | 0.023372  | 0.028527  | 0.004322  | -0.095333 | -0.116516 | 0.00492   | 0.022961  | 0.03148   | 0.0         | 2024-05-12 05:48:41 |
| 9997 | -0.068554 | 0.044618  | 0.029291  | 0.063244  | 0.003076  | 0.044868  | -0.028499 | -0.047499 | -0.0164   | -0.01148  | 0.0         | 2024-05-12 05:48:44 |
| 9998 | 0.045825  | 0.083247  | 0.0       | 0.0       | 0.0       | 0.0       | 0.0       | 0.0       | 0.0       | 0.0       | 0.0         | 2024-05-12 05:48:47 |
| 9999 | 0.005295  | 0.020722  | 0.101481  | 0.180408  | 0.02918   | 0.056719  | 0.045378  | 0.063938  | 0.032738  | 0.064364  | 0.0         | 2024-05-12 05:48:49 |

10000 rows × 12 columns

### Data Gathering:

From Coinbase Exchange, using API, we fetched LOB data for all 3 trading pairs and 10000 observations for each, which took substantial computational resources. For example, to fetch 100 limit order book observations just for one pair “BTC-USD” took approximately 3 minutes. Therefore, it would take 5\*3 hours to fetch all necessary data unless we would run it on cloud in parallel.

We also extracted Open, Close, High, Low, Volume (OCLHV) data, for the same timestamps as LOB data to integrate this with order flow and imbalance. The **granularity** for OCLHV data for each trading pair was chosen equal to 60, which is **1 minutes** and approximately **2240 observations** in 1 trading day. For LOB data we fetched about **10,000 observations** for each trading pair over the course of a **day** as well. This approach provides a robust dataset incorporating both traditional market indicators and advanced metrics capturing market dynamics. The addition of OF and OFI aims to enhance the model's ability to discern underlying market pressures and trends that may not be evident from price data alone. The time series graphs with candles for each cryptocurrency pair for a chosen day can be found in Appendix 2.

By integrating these stationary transformations into the predictive models, we could assess their impact on forecasting performance through backtesting. The expectation is that these metrics will offer predictive insights into market directionality, improving the accuracy and reliability of trading strategy. This methodological shift towards stationary transformations of LOB data signifies a more nuanced understanding of market mechanics, potentially leading to superior forecasting models that can adapt to and capitalize on the fluid dynamics of the cryptocurrency markets.

### The model:

The exploration of ensemble and hybrid models has emerged as a promising direction for improving prediction accuracy. By combining the strengths of multiple predictive models, ensemble approaches can offer more robust and accurate forecasts. For example, the use of ensemble models incorporating LSTM, GRU, and other DL architectures has been shown to marginally improve prediction performance (Murray et al., 2023).

**Table 4. Ensemble and Hybrid models Comparison**

| Ensemble               | RMSE   | MAE    | MAPE  | R2   |
|------------------------|--------|--------|-------|------|
| LSTM                   | 0.0222 | 0.0173 | 3.86% | 0.73 |
| GRU, LSTM              | 0.0225 | 0.0174 | 3.89% | 0.73 |
| HYBRID, LSTM           | 0.0225 | 0.0174 | 3.89% | 0.73 |
| HYBRID, GRU, LSTM      | 0.0226 | 0.0175 | 3.90% | 0.73 |
| LSTM, KNN              | 0.0227 | 0.0175 | 3.92% | 0.73 |
| GRU, LSTM, KNN         | 0.0227 | 0.0176 | 3.91% | 0.72 |
| GRU, LSTM, TCN         | 0.0227 | 0.0176 | 3.92% | 0.72 |
| LSTM, TCN              | 0.0227 | 0.0176 | 3.93% | 0.72 |
| HYBRID, LSTM, KNN      | 0.0227 | 0.0175 | 3.92% | 0.72 |
| HYBRID, GRU, LSTM, KNN | 0.0227 | 0.0175 | 3.91% | 0.72 |

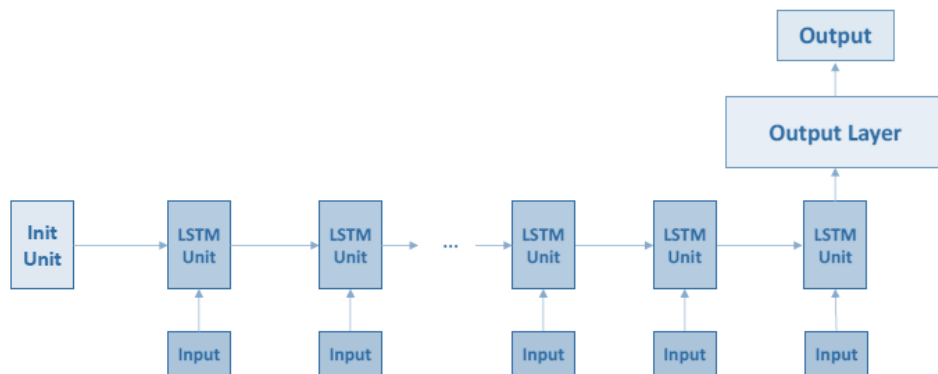
Despite the promising outcomes associated with these models, challenges such as data overfitting, model complexity, and the dynamic nature of financial markets persist.

These issues not only complicate model training and validation but also highlight the importance of continuous model adaptation and the exploration of diverse data sources, including social media for a more comprehensive sentiment analysis (Zeng & Jiang, 2023; Koker & Koutmos, 2020). Moreover, the inherent volatility and unpredictability of the market, coupled with the influence of external factors such as regulatory changes and market sentiment, complicate the forecasting task. Furthermore, the computational complexity and data requirements of advanced DL models pose practical limitations.

Therefore, we are proposing a long short-term memory (LSTM) network to predict the future direction of an order book at high frequency. Using Coinbase order book data for Bitcoin, Ethereum and Solana our goal is to forecast whether the mid-price will rise, fall, or remain constant in the subsequent time series observation.

The universal approximation theorem suggests that a feedforward neural network with just one hidden layer has the capacity to approximate any continuous function within a defined compact subset. However, specialized neural network designs can tap into the unique patterns present in data—a feature especially beneficial for analyzing complex financial time series. Long Short-Term Memory networks (LSTMs) architecture has the ability to process and adapt to data over time in ways that enhance their performance in challenging modeling tasks. In contrast, simpler feedforward networks may face difficulties with instability and a lack of clarity when trying to decipher intricate data relationships.

The exact number of layers will be determined through hyperparameter optimization – to effectively capture the temporal sequences within the market data. These networks will be trained using advanced optimizer Adam to refine learning rates and mitigate the vanishing gradient problem inherent in deep network training.



**Figure 3. LSTM model architecture**

LSTMs, a subset of recurrent neural networks, are adept at managing sequences and their inherent temporal correlations, countering the vanishing gradient dilemma that plagues standard RNNs. An LSTM unit, equipped with a memory cell and three regulatory gates, meticulously manages information flow, deciding what data is stored, updated, or forgotten, thus allowing for efficient processing of temporal sequences in data.

$$\begin{aligned}
f_t &= \sigma(U^f x_t + W^f h_{t-1} + b^f) \\
i_t &= \sigma(U^i x_t + W^i h_{t-1} + b^i) \\
o_t &= \sigma(U^o x_t + W^o h_{t-1} + b^o) \\
c_t &= f_t \circ c_{t-1} + i_t \circ \tanh(U^c x_t + W^c h_{t-1} + b^c) \\
h_t &= o_t \circ \tanh(c_t)
\end{aligned} \tag{4}$$

where  $m$  is the number of LSTM units in the module,  $\sigma(x) = \frac{1}{1+e^{-x}}$  is the sigmoid activation function,  $f_t \in \mathbb{R}^{m \times f}$  is the forget gate's activation vector,  $i_t \in \mathbb{R}^{m \times i}$  is the input gate's activation vector,  $o_t \in \mathbb{R}^{m \times o}$  is the output gate's activation vector,  $c_t \in \mathbb{R}^{m \times c}$  is the LSTM unit's hidden state vector, and  $h_t \in \mathbb{R}^{m \times h}$  is the unit's output vector.  $U \in \mathbb{R}^{m \times n}$ ,  $W \in \mathbb{R}^{m \times m}$ , and  $b \in \mathbb{R}^{m \times b}$  are learned during training and represent the weight matrices in connection to the input vector, the weight matrices in connection to the previous output state, and the bias vectors, respectively (Kolm, Turiel, & Westray, 2021).

Below are the architecture and the results of our first constructed LSTM model, that is incorporating Batch Normalization and training with class weights. We normalized the output of a previous activation layer by subtracting the batch mean and dividing by the batch standard deviation, then scaling and shifting using learned parameters. This helps to stabilize and accelerate the training process by reducing internal covariate shift. It also acts as a regularizer, potentially improving the model's ability to generalize to new data. By applying class weights, we improved the balance and performance of the model on the "Down" and "Up" classes. These improvements contributing to training stability and potentially better performance on underrepresented classes.

We also explored Vector autoregression (VAR) model to fit order flow (OF) and order flow imbalance (OFI) data. It's important to determine the correct number of past observations, or "lags," to include in the model. This helps ensure that LSTM model can adequately capture the dynamics of the data.

Model: "functional\_1"

| Layer (type)                               | Output Shape   | Param # |
|--------------------------------------------|----------------|---------|
| input_layer (InputLayer)                   | (None, 17, 20) | 0       |
| dropout (Dropout)                          | (None, 17, 20) | 0       |
| lstm (LSTM)                                | (None, 17, 64) | 21,760  |
| batch_normalization (BatchNormalization)   | (None, 17, 64) | 256     |
| dropout_1 (Dropout)                        | (None, 17, 64) | 0       |
| lstm_1 (LSTM)                              | (None, 64)     | 33,024  |
| batch_normalization_1 (BatchNormalization) | (None, 64)     | 256     |
| dropout_2 (Dropout)                        | (None, 64)     | 0       |
| dense (Dense)                              | (None, 3)      | 195     |

Total params: 55,491 (216.76 KB)

Trainable params: 55,235 (215.76 KB)

Non-trainable params: 256 (1.00 KB)

Model: "functional\_3"

| Layer (type)                               | Output Shape   | Param # |
|--------------------------------------------|----------------|---------|
| input_layer_1 (InputLayer)                 | (None, 61, 10) | 0       |
| dropout_3 (Dropout)                        | (None, 61, 10) | 0       |
| lstm_2 (LSTM)                              | (None, 61, 64) | 19,200  |
| batch_normalization_2 (BatchNormalization) | (None, 61, 64) | 256     |
| dropout_4 (Dropout)                        | (None, 61, 64) | 0       |
| lstm_3 (LSTM)                              | (None, 64)     | 33,024  |
| batch_normalization_3 (BatchNormalization) | (None, 64)     | 256     |
| dropout_5 (Dropout)                        | (None, 64)     | 0       |
| dense_1 (Dense)                            | (None, 3)      | 195     |

Total params: 52,931 (206.76 KB)

Trainable params: 52,675 (205.76 KB)

Non-trainable params: 256 (1.00 KB)

**Figure 4. LSTM\_1 model architecture for OF/OFI classification**

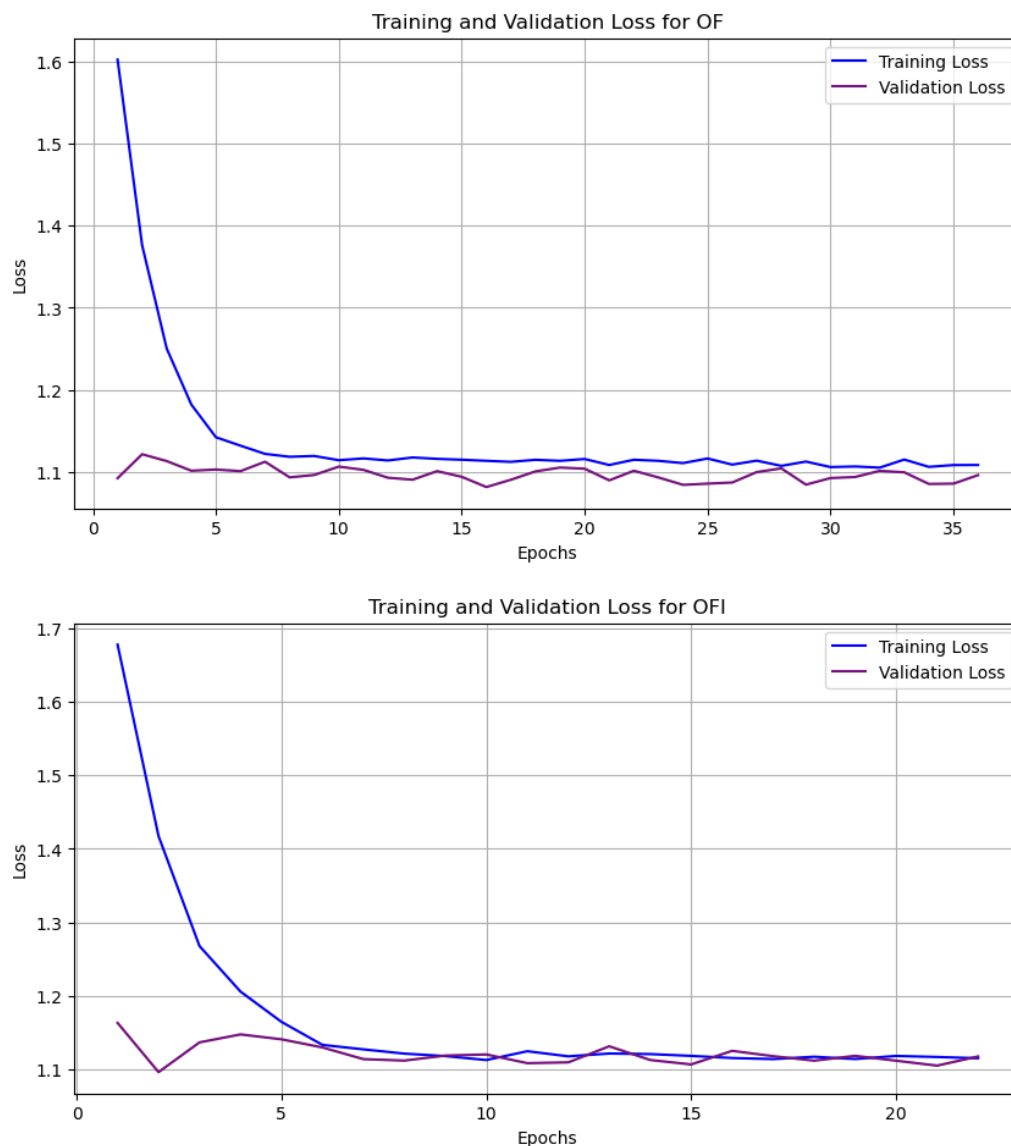
|              |           |        |          |         |      |                      |
|--------------|-----------|--------|----------|---------|------|----------------------|
| OF           |           |        |          |         |      |                      |
|              | precision | recall | f1-score | support |      |                      |
| Down         | 0.16      | 0.39   | 0.22     | 270     | True | OF Confusion Matrix  |
| Stationary   | 0.74      | 0.30   | 0.42     | 1415    |      |                      |
| Up           | 0.17      | 0.43   | 0.25     | 298     |      |                      |
| micro avg    | 0.33      | 0.33   | 0.33     | 1983    |      |                      |
| macro avg    | 0.36      | 0.37   | 0.30     | 1983    |      |                      |
| weighted avg | 0.58      | 0.33   | 0.37     | 1983    |      |                      |
| samples avg  | 0.33      | 0.33   | 0.33     | 1983    |      |                      |
| OFI          |           |        |          |         | True | OFI Confusion Matrix |
|              | precision | recall | f1-score | support |      |                      |
| Down         | 0.18      | 0.30   | 0.22     | 263     |      |                      |
| Stationary   | 0.73      | 0.36   | 0.48     | 1380    |      |                      |
| Up           | 0.18      | 0.49   | 0.26     | 296     |      |                      |
| micro avg    | 0.37      | 0.37   | 0.37     | 1939    |      |                      |
| macro avg    | 0.36      | 0.38   | 0.32     | 1939    |      |                      |
| weighted avg | 0.57      | 0.37   | 0.41     | 1939    |      |                      |
| samples avg  | 0.37      | 0.37   | 0.37     | 1939    |      |                      |

**Figure 5. LSTM\_1 model results and confusion matrix**

From Figure 5 we can see the classification metrics - the precision, recall, F1-score, and support for each class in both OF and OFI classification tasks. These metrics provide a detailed picture of the model's performance across different classes. Precision, which measures the accuracy of the positive predictions, shows variability across classes. For instance, in the OF classification, the 'Stationary' class achieves higher precision compared to the 'Down' and 'Up' classes. This indicates that the model is more confident and accurate when predicting the 'Stationary' state, but less so for 'Down' and 'Up' states. Recall, on the other hand, measures the model's ability to identify all positive instances of a class. Here, again, the 'Stationary' class shows better recall, meaning the model is more successful in identifying instances of 'Stationary' compared to the other states.

The F1-score, which is the harmonic mean of precision and recall, balances these two metrics and provides a single measure of performance. For both OF and OFI classifications, the F1-scores indicate that while the model performs reasonably well for the 'Stationary' class, it struggles more with the 'Down' and 'Up' classes. This might be due to the inherent imbalance in the data, as indicated by the support values, which show that 'Stationary' has a significantly higher number of instances compared to 'Down' and 'Up'. The support values highlight the number of true instances for each class, and the imbalance can skew the model's performance, favoring the more frequent 'Stationary' class.

Analyzing the confusion matrices showing the distribution of predictions across the actual classes, we can see that the majority of misclassifications occur between 'Stationary' and the other states. This, again, suggests that the model often predicts 'Stationary' when the true class is 'Down' or 'Up', which aligns with the higher precision and recall for 'Stationary'. Addressing this misclassification could involve further tuning the model or incorporating additional features that help better distinguish between these states.



**Figure 6. LSTM\_1 model Training and Validation Losses**

From the graphs in Figure 6, we can see the learning process of the LSTM model which indicates a successful training process where both training and validation losses decrease over time. The validation loss follows the training loss closely, suggesting that the model is generalizing well to unseen data without significant overfitting. The first graph depicts the model's loss over epochs for both the training set and the validation set. Ideally, we want to see both lines trending downwards, with the validation loss following closely behind the training loss, which seems to be the case here after the initial epochs.

The training and validation loss both show some volatility, which is normal in the beginning as the model starts to learn, but since the volatility continues, it might suggest that the learning rate is too high or that the model's capacity (its complexity) is not adequately tuned to capture the underlying patterns in the data.

The reasonable performance of the model might be due to the fact that it's too simplistic to do justice to the underlying complexity of the data. We will consider more LSTM layers or increasing the number of units in the LSTM layers, as well as adjusting the model's hyperparameters, such as the learning rate, batch size, or epoch. We are going to optimize the model using the GridSearch.

Alongside the LSTM model, we propose Variational Autoencoder (VAE) Variational Autoencoder (VAE) for risk estimation. This approach is particularly suited for capturing the latent representation of high-dimensional financial data and subsequently estimating the covariance matrix. The VAE processes high-dimensional order flow (OF) and order flow imbalance (OFI) data, reducing noise and highlighting critical features within the data.

The VAE addresses the limitations of traditional linear factor models by leveraging neural networks to model complex, non-linear relationships within the data. This allows for a more accurate estimation of risk in financial portfolios. Once the VAE compresses the data into its latent representation, we use this representation to estimate the covariance matrix, which is crucial for portfolio risk management. This covariance matrix is then used in the construction of a minimum variance portfolio, demonstrating the practical application of the VAE in financial scenarios.

Following the transformation and risk estimation, the encoded data from the VAE is fed into a Long Short-Term Memory (LSTM) network for trading purposes. The LSTM utilizes the refined features from the VAE to make predictions about future time steps, enhancing its sequential modeling capabilities. This two-step process leverages the strengths of both the VAE and LSTM: the VAE excels at feature extraction and noise reduction, while the LSTM is adept at recognizing temporal patterns.

The process involves first training the VAE on the OF and OFI datasets to create the latent representation. This representation is then used to estimate the covariance matrix. The estimated covariance matrix is crucial for constructing a minimum variance portfolio, which is a key application in risk management. Those graphs can be found in Appendix 4 to 7.

For optimization, we performed a grid search to identify the most effective model parameters. The best-performing parameters for the Variational Autoencoder (VAE) used with the Order Flow Imbalance (OFI) data were as follows: a hidden dimension of 128, a latent dimension of 10, a learning rate of 0.0001, a batch size of 32, and 100 epochs. These parameters were determined through extensive experimentation, ensuring the robustness and effectiveness of the model in handling high-dimensional financial data. A hidden dimension of 128 was chosen to capture the complex relationships in the data, while a latent dimension of 10 suggests that the model's latent space, where it encodes the input data, consists of 10 features. This helps in capturing the essence of the input data while maintaining computational efficiency.

By adjusting the batch size and epochs, we experimented with the learning process to find an optimal balance between training time and model performance. Our model's loss function, which includes both reconstruction loss and KL divergence, is finely tuned to balance the model's focus between data fidelity and latent space regularization.

In our Variational Autoencoder (VAE) model, the Kullback-Leibler (KL) divergence is a crucial component. It's part of the loss function and helps the model learn a well-formed latent space that approximates the prior distribution. The KL divergence acts as a regularizer in the loss function of a VAE. It measures how much one probability distribution diverges from a second, expected probability distribution. In the context of VAEs, it's used to compare the distribution of the latent variables encoded by the encoder network to the prior distribution.

The loss function of a VAE defines as follows and is composed of two parts:

$$L_{total} = L_{recon} + \beta \cdot D_{KL}(q(z|x) || p(z)) \quad (5)$$

where  $L_{recon}$  is a Reconstruction Loss that measures how well the decoded samples match the original inputs. In the case of an LSTM-VAE this would be the Mean Squared Error (MSE) between the predicted sequence and the true sequence:

$$L_{recon} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2 \quad (6)$$

KL Divergence part of the loss function measures how the learned latent variable distribution (encoded by the LSTM encoder) diverges from the prior distribution. It encourages the encoder to produce latent variables that follow the prior distribution, which helps in generating more consistent and coherent data points from the latent space during the decoding process. The KL divergence part of the loss ensures that the latent variables  $z$  follow the prior distribution  $p(z)$ , which is often chosen to be the standard normal distribution  $N(0, I)$ . The encoder of the VAE models the posterior distribution  $q(z|x)$  as a normal distribution  $N(\mu, \sigma^2 I)$  with a mean  $\mu$  and standard deviation  $\sigma$  that are functions of the input  $x$ . The KL divergence between  $q(z|x)$  and  $p(z)$  is computed as follows:

$$D_{KL}(q(z|x) || p(z)) = -\frac{1}{2} \sum_{j=1}^J (1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2) \quad (7)$$

where  $J$  is the dimensionality of the latent space.

The KL divergence is integrated into the training of the VAE by adding it to the reconstruction loss to form the final loss that is minimized during training. This integration is necessary for the VAE to not only fit the data but also ensure that the latent space has good properties, enabling generative processes. By utilizing MSE for the reconstruction loss and combining it with KL divergence, we aim to maintain a balance between the model's ability to reconstruct input data and its generalization capability. By using these optimized parameters and carefully tuning the loss function, we ensured that our VAE models for both OFI and price/sentiment data were robust and effective in capturing the essential features of high-dimensional financial data.

The second part of the project is to integrate sentiment analysis by employing FinBERT, a transformer-based model pre-trained on financial texts. It will be fine-tuned on domain-specific datasets to accurately capture market sentiment, which will serve as a pivotal input feature for the predictive models.

"Financial Sentiment Analysis using FinBERT with Application in Prediction Stock Movement" by Qingyun Zeng and Tingsong Jiang paper explores the innovative Application of FinBERT, a variant of BERT trained specifically on financial data, which is adept at capturing the nuances of financial sentiment. This specialization could potentially offer superior sentiment analysis in the financial context compared to generic language models. Using the Cryptopanic API key, we are able to fetch financial news for each Crypto Pair. For text summarization, we used models that are trained specifically for that task, such as BART (Bidirectional and Auto-Regressive Transformers).

|    | ticker | summary                                            | sentiment |
|----|--------|----------------------------------------------------|-----------|
| 0  | BTC    | Former President Donald Trump's Advisers Explo...  | 0         |
| 1  | BTC    | Summarize: Bitcoin's range-bound action puts e...  | 0         |
| 2  | BTC    | Charlie Lee on Privacy & MWEB (Mimble Wimbles E... | 0         |
| 3  | BTC    | Analysts claim Bitcoin Topped and Could Drop t...  | -1        |
| 4  | BTC    | Ethereum prices Soar as Transaction Fees Hit S...  | 1         |
| 5  | BTC    | Analyst Eyes \$300K Bitcoin Price as BTC Approa... | 0         |
| 6  | BTC    | summarize: Is The 'Memecoin Supercycle' Over? ...  | 0         |
| 7  | BTC    | Shiba Inu is one of the most popular dogs in t...  | 0         |
| 8  | BTC    | summarize: Bitcoin in a few months after the h...  | 0         |
| 9  | BTC    | BlackRock Bitcoin ETF Faced Zero Inflows for 3...  | -1        |
| 10 | BTC    | Crypto Trader says Bitcoin Presenting 'Generat...  | 0         |
| 11 | BTC    | summarize: Solana Advances Congestion-Alleviat...  | 0         |
| 12 | BTC    | summarize: Bitcoin Primed For November 2020-Es...  | 0         |
| 13 | BTC    | Summarize: Bitcoin & Altcoins Suffer Price Dro...  | -1        |
| 14 | BTC    | Hong Kong's Spot Crypto ETF Launch: No Gateway...  | 0         |
| 15 | BTC    | Summarize: Four Reasons Why Ethereum Is Not a ...  | 0         |
| 16 | BTC    | summarize: Render Revving Up: Analyst Predicts...  | 1         |
| 17 | BTC    | summarize: A Major WARNING Sign for Bitcoin fr...  | -1        |
| 18 | BTC    | XRP Getting Absolutely Crushed by Bitcoin Desp...  | 1         |
| 19 | BTC    | Amir Taaki: 'Globalist Power Is No Longer a Mo...  | 0         |
| 20 | ETH    | Einsteinium is a decentralised, peer-to-peer d...  | 0         |
| 21 | ETH    | Ethereum prices Soar as Transaction Fees Hit S...  | 1         |
| 22 | ETH    | Summarize: Justin Sun speaks on challenges fac...  | 1         |
| 23 | ETH    | Einstein: ETH's run to \$3.3K may be false brea... | 0         |

**Figure 7. Example of DataFrame with Sentiment output**

Alongside with fetched previously OCLHV data we now constructed dataframe for BTC\_USD prices with sentiment score from FinBERT and merged by timestamps. The next step was to add a NSI index as well, as discussed in Zeng & Jiang paper - “we proposed a numerical sentiment index based on the price movement, which is defined as  $NSI = 1$  if  $return > s$  and  $NSI = 0$  if  $-s < return < s$  and  $NSI = -1$  if  $return < -s$ ” (Zeng & Jiang, 2023). The formula for calculating returns is given by the ratio of the closing price minus the opening price to the opening price. This formula can be extended to calculate the return over any number of days  $k$  by using the closing price on the  $k$ -th day after the current one.

$$return = \frac{ClosePrice_{t+k} - OpenPrice_t}{OpenPrice_t} \quad (8)$$

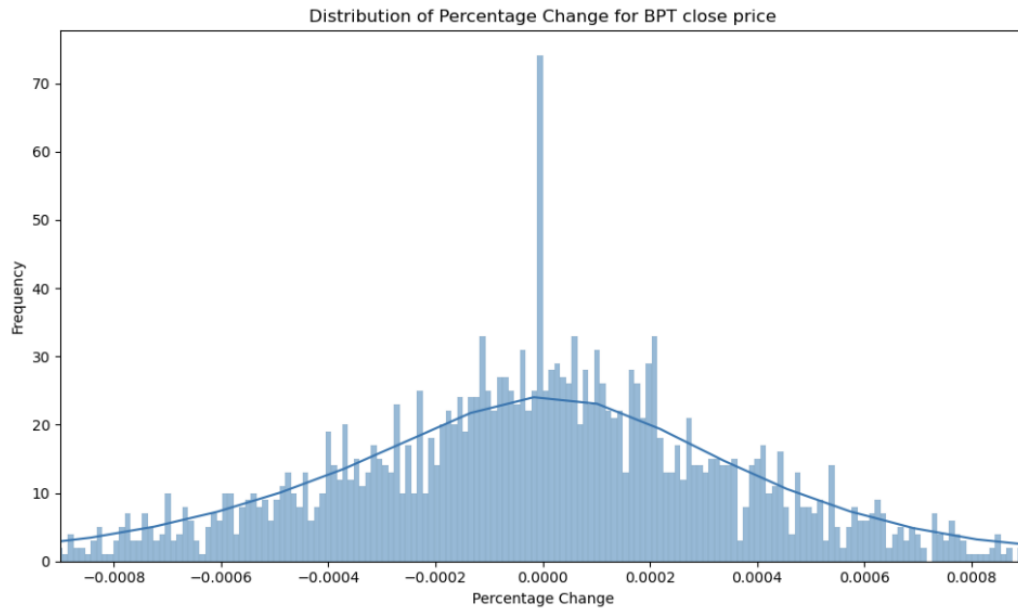
The Normalized Sentiment Indicator (NSI) plays a role in refining the predictions of closing prices using models like FinBERT, which assess the interplay between textual sentiment and market dynamics. To forecast closing prices while mitigating issues like long-term inflation and stock diversity, we apply a MinMax scaler to the data before modeling. Our data is split into a 90% training set and a 10% testing set. Predicting long-term market trends is challenging due to unpredictable events.

**Table 5. DataFrame with NSI and Sentiment scores**

|   | time                      | low      | high     | open     | close    | volume    | sentiment | return    | NSI |
|---|---------------------------|----------|----------|----------|----------|-----------|-----------|-----------|-----|
| 0 | 2024-04-27 17:00:00-04:00 | 63208.37 | 63248.31 | 63239.09 | 63243.87 | 5.091023  | 0.0       | 0.000076  | 1   |
| 1 | 2024-04-27 17:05:00-04:00 | 63162.04 | 63254.80 | 63243.86 | 63168.15 | 4.158068  | 0.0       | -0.001197 | -1  |
| 2 | 2024-04-27 17:10:00-04:00 | 63116.59 | 63172.43 | 63165.18 | 63118.92 | 8.945956  | 0.0       | -0.000732 | -1  |
| 3 | 2024-04-27 17:15:00-04:00 | 63105.69 | 63200.26 | 63119.80 | 63166.36 | 9.202459  | 0.0       | 0.000738  | 1   |
| 4 | 2024-04-27 17:20:00-04:00 | 63115.23 | 63199.47 | 63163.41 | 63161.46 | 21.792795 | 0.0       | -0.000031 | 0   |

The dynamic nature of market sentiment, especially in response to real-time events, poses a challenge for any model relying on historical data for prediction. The lag between news publication and market reaction may limit the model's effectiveness in fast-moving markets (Zeng & Jiang, 2023).

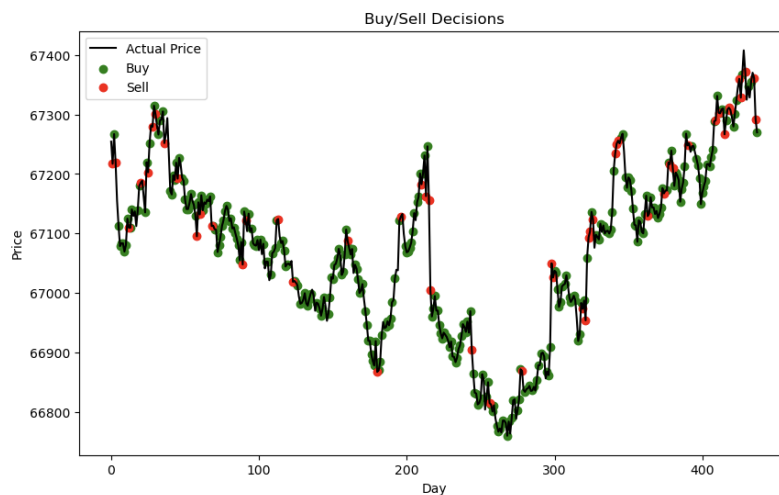
Below is the distribution of the return for BTC close price, which shows a pronounced peak at zero. This distribution suggests that small percentage changes in BTC's close price are most common, reflecting frequent minor fluctuations in price. The sharp peak at zero indicates a high frequency with short time frames. The fat tails indicate that while extreme changes in percentage are less frequent, they are still significant enough to affect the distribution, highlighting the inherent volatility of BTC prices. The symmetry of the distribution also suggests that upward and downward movements are equally probable, reflecting the dynamic and unpredictable nature of cryptocurrency markets.



**Figure 8. Distribution of Returns for BTC time series**

We trained LSTM models using historical BTC price data combined with sentiment scores and NSI values, which lead to potentially more informed trading decisions, predicting future price movements based on these features. Similarly, to include a comprehensive risk analysis, we trained a VAE model for the price/sentiment data with the same best-performing parameters: a hidden dimension of 128, a latent dimension of 10, a learning rate of 0.0001, a batch size of 32, and 100 epochs. These parameters were determined through grid search and extensive experimentation, ensuring that the model effectively encodes the input data into a lower-dimensional space with 10 features. The learning rate of 0.0001 ensures a smooth convergence during the optimization process.

Number of buy signals: 303  
Number of sell signals: 51



**Figure 9. Buy/Sell Decisions on BTC Train Data**

The "Buy/Sell Decisions" graph in Figure 9 illustrates the actual BTC price movements over a period of days, with buy and sell signals generated by our LSTM model. The black line represents the actual price, while green and red markers indicate buy and sell signals, respectively, showing 303 buy signals and 51 sell signals.

Now we have two tuned and trained separate LSTM models: one on historical BTC price data with sentiment scores and NSI values, and the other on OFI data. We then combined the predictions from both models to enhance the robustness of the trading signals. Additionally, we employed two VAE models, one trained on the OFI dataset and the other on the price/sentiment dataset, to estimate the risk associated with each trading decision. The combined predictions and risk metrics were used to generate buy, sell, and hold signals, which were then backtested on the test data to evaluate the performance. This integrated approach, leveraging both price/sentiment and OFI data along with risk management from the VAEs, enabled us to create a more comprehensive and effective trading strategy.

Through integrating the predictability of prices and the dynamics of the limit order book, reinforcement learning surpasses numerous established methods, including those based on price prediction through supervised learning. For reinforcement learning, two strategies exist for training agents with market data: employing offline data and utilizing online data. Offline data use involves processing historical market data for training, which is more time-efficient than the use of online data, which relies on real-time market data. Since the supervised learning component necessitates historical data storage, offline data is accessible for simulating market conditions for learning purposes. The implementation includes developing the backtesting procedure, utilizing the following algorithm (Jaddu and Bilokon, 2023):

```

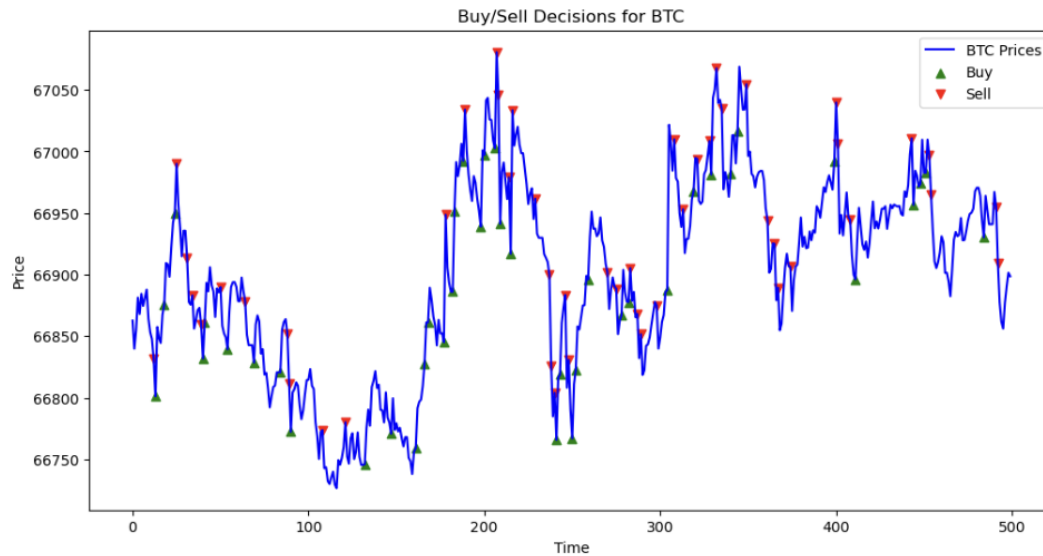
1 Load agent  $A$ 
2 Load backtesting data  $D$  as a list of tuples with form  $(OFI, Mid-price)$ 
3 Initialise empty trade log  $T$ 
4 Initialise previous action  $a' \leftarrow NULL$ 
5 Initialise current trade  $t \leftarrow NULL$ 
6 Initialise current state  $s \leftarrow next(D)$ 
7 repeat
8    $a \leftarrow A(s)$  # buy or sell signal
9   if  $a = a'$  then
10    | Update  $t$  using  $s$ 
11  end
12  else
13    | Append non-NULL  $t$  to  $T$ 
14    | Reinitialise  $t$  with  $(s, a)$  and transaction costs
15  end
16   $s \leftarrow next(D)$ 
17   $a' \leftarrow a$ 
18 until  $D$  is fully iterated;
19 Use  $T$  to calculate performance metrics

```

**Figure 10. Backtesting Algorithm (Jaddu and Biloko, 2023)**

We conducted backtesting to offer the agent subsequent state and reward based on the current state and action while incorporating transaction costs into each trade's profit or

loss. Evaluating trading strategies or automated trading bots in a risk-free yet realistic setting is crucial before their live trading deployment with real capital. Backtesting simulates market conditions using historical data to assess the outcomes of buy/sell signals as if they were actual trades. These results help gauge the agent's performance post-simulation. Since Coinbase does not offer historical limit order book (LOB) data, backtesting required independent coding. The backtesting routine alternates the agent between maintaining active buy or sell positions, with all trade signals and outcomes recorded for performance evaluation. Although we did not use OpenAI's Gym Environment or incorporate retail-level commissions into each trade's profit or loss, we accounted for transaction costs in the assessment.



**Figure 11. Backtesting Results**

The graph above "Buy/Sell Decisions for BTC" illustrates the results of our backtesting. It shows BTC price movements over time with buy signals (green triangles) and sell signals (red triangles) overlaid. The model's signals are based on a combination of LSTM predictions and risk assessments from VAEs trained on OFI and price/sentiment data. The initial balance was set at 5000 units, and the final balance post-backtesting was 6933.93 units, indicating a profitable strategy. However, the highly precise signals might indicate potential data leakage, which warrants further investigation and validation. Additionally, since Solana (SOL) and Ethereum (ETH) are highly correlated with Bitcoin (BTC), we applied the same strategies and models in parallel to these assets. The detailed results of these applications can be found in Appendix 7.

Future developments will involve forward testing, which considers the viability of a trading strategy based on its backtesting performance. This involves connecting the agent to a platform like Coinbase for live, unseen data processing and real order submissions. Forward testing, essential for accurate simulation, will utilize the Coinbase API through a Flask-based web application, enabling model-based signal transmission to the platform via POST requests. It supports both real and simulated (paper trading) money, providing a more authentic experience as the platform updates the agent on order executions, including any discrepancies from expected entry prices (slippage) and variable spreads. Forward testing's

outcomes offer a more precise reflection of a strategy's effectiveness compared to backtesting. Despite the higher accuracy, forward testing is more resource-intensive, relying on live data rather than historical data iterations, explaining why backtesting remains a valuable preliminary step (Jaddu and Biloko, 2023).

```

1 Load agent  $A$ 
2 Acquire live data feed  $s$  as a tuple ( $OFI$ ,  $Mid-price$ )
3 Initialise empty trade log  $T$ 
4 Initialise previous action  $a' \leftarrow NULL$ 
5 repeat
6    $a \leftarrow A(s)$  # buy or sell signal
7   if  $a$  is not  $a'$  then
8     Append result of  $t$  if it exists to  $T$ 
9     Close position  $t$  if it exists and open position  $t$  aligned with  $a$ 
10  end
11  Wait for an update from the live data feed and store the new state in  $s$ 
12   $a' \leftarrow a$ 
13 until terminated;
14 Use  $T$  to calculate performance metrics

```

**Figure 12. Forward Testing Algorithm (Jaddu and Biloko, 2023)**

Furthermore, in future developments, we can consider combining these assets as a portfolio for a more holistic strategy, although their actual correlation and combined performance need to be thoroughly analyzed. Optimizing hyperparameters can be approached in various ways, yet given sufficient time for tuning, we employed a straightforward and effective method grid search. This method methodically explores all possible combinations of hyperparameters, selecting the one with the highest validation performance.

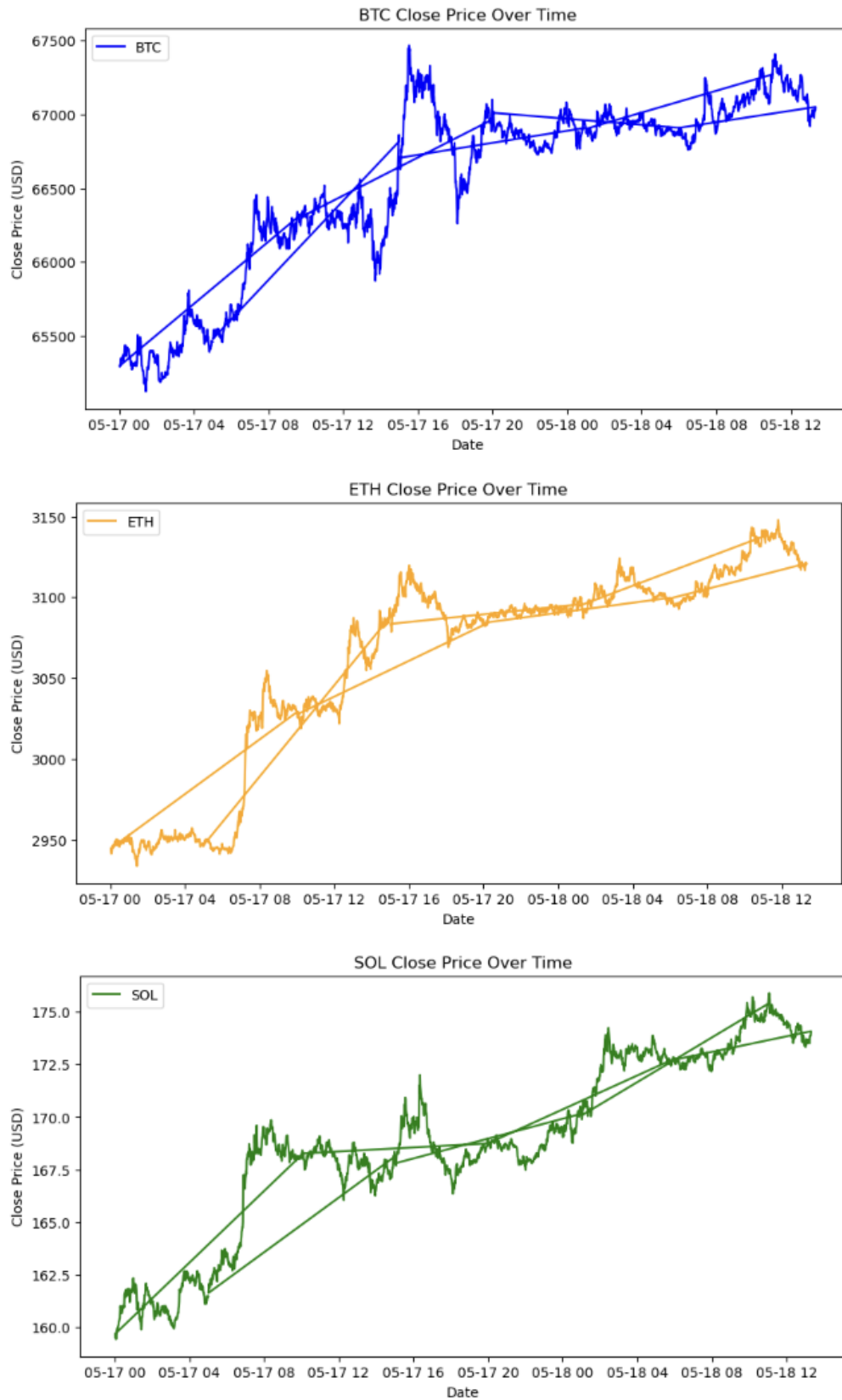
Collectively, combining sentiment analysis and VAEs for risk estimation with LSTM for predicting future direction of prices is a robust methodological approach. It blends the strengths of deep learning in processing sequential data with the nuanced understanding of market sentiment, offering a comprehensive tool for market analysis. These goals are directed towards creating an integrated machine learning framework that not only predicts market movements but also assesses risks and considers market sentiment to inform cryptocurrency trading decisions. This capstone project transcends theoretical academic research and ventures into the creation of a practical, professional tool designed to fortify the synergy between state-of-the-art computational methodologies and the elusive nature of cryptocurrency trading.

## **Appendix 1**

The project GitHub Repository with Python Notebook is [here](#)

## Appendix

2



Appendix

3

|              |           |        |          |         |
|--------------|-----------|--------|----------|---------|
| OF           |           |        |          |         |
|              | precision | recall | f1-score | support |
| Down         | 0.16      | 0.39   | 0.22     | 270     |
| Stationary   | 0.74      | 0.30   | 0.42     | 1415    |
| Up           | 0.17      | 0.43   | 0.25     | 298     |
| micro avg    | 0.33      | 0.33   | 0.33     | 1983    |
| macro avg    | 0.36      | 0.37   | 0.30     | 1983    |
| weighted avg | 0.58      | 0.33   | 0.37     | 1983    |
| samples avg  | 0.33      | 0.33   | 0.33     | 1983    |
| OFI          |           |        |          |         |
|              | precision | recall | f1-score | support |
| Down         | 0.18      | 0.30   | 0.22     | 263     |
| Stationary   | 0.73      | 0.36   | 0.48     | 1380    |
| Up           | 0.18      | 0.49   | 0.26     | 296     |
| micro avg    | 0.37      | 0.37   | 0.37     | 1939    |
| macro avg    | 0.36      | 0.38   | 0.32     | 1939    |
| weighted avg | 0.57      | 0.37   | 0.41     | 1939    |
| samples avg  | 0.37      | 0.37   | 0.37     | 1939    |

OF Confusion Matrix

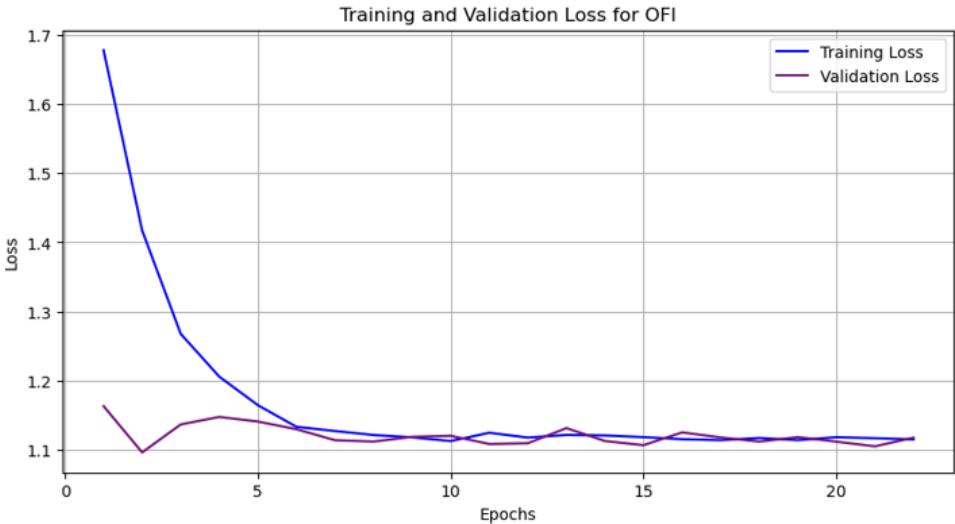
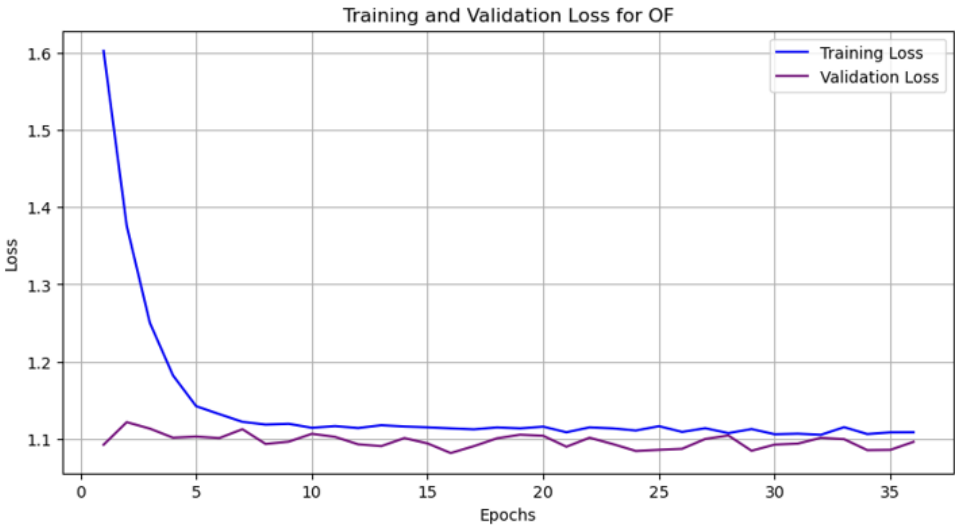
|            |      |            |     |
|------------|------|------------|-----|
|            | Down | Stationary | Up  |
| Down       | 132  | 52         | 86  |
| Stationary | 490  | 355        | 570 |
| Up         | 106  | 68         | 124 |

Predictions

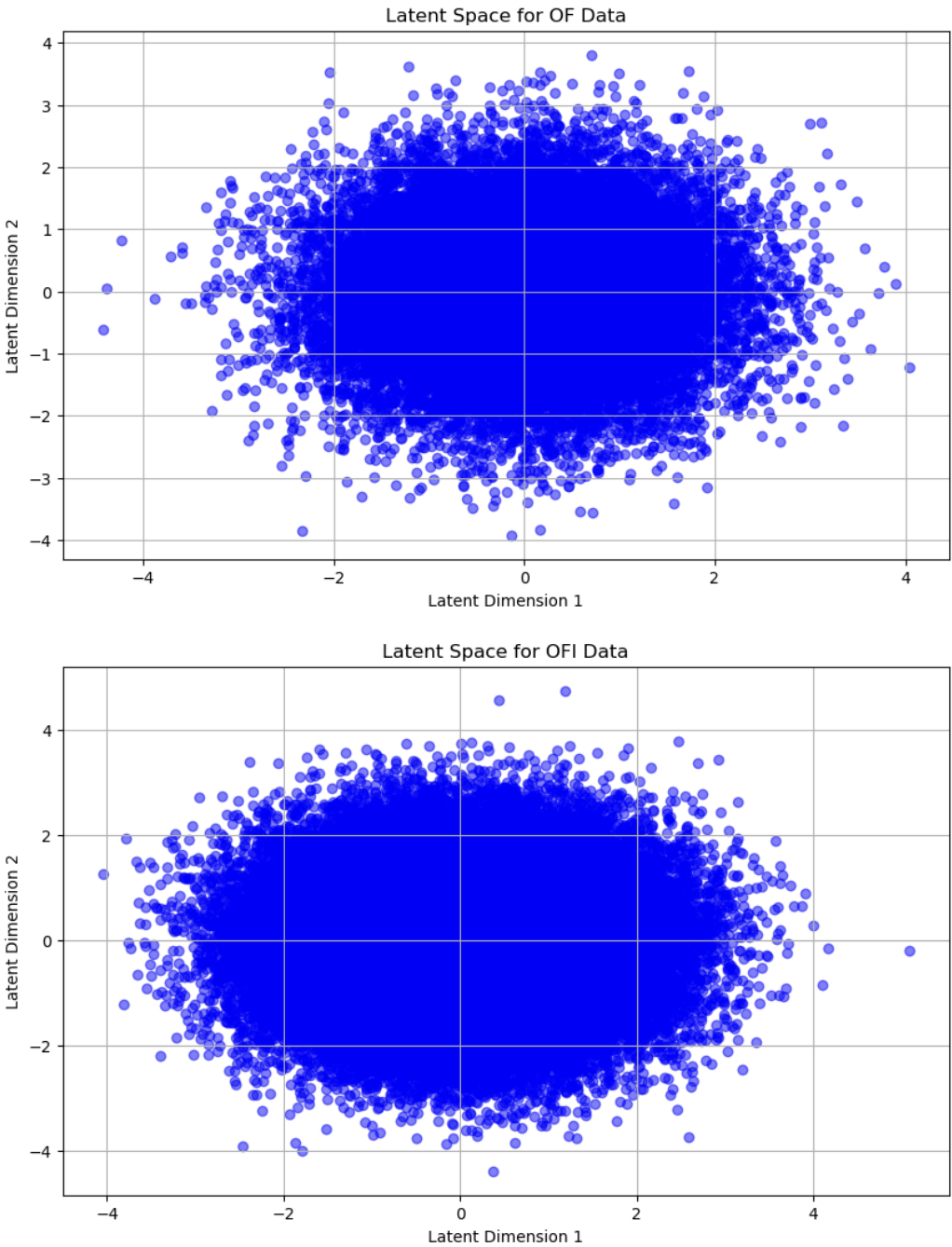
OFI Confusion Matrix

|            |      |            |     |
|------------|------|------------|-----|
|            | Down | Stationary | Up  |
| Down       | 102  | 70         | 91  |
| Stationary | 347  | 532        | 501 |
| Up         | 74   | 97         | 125 |

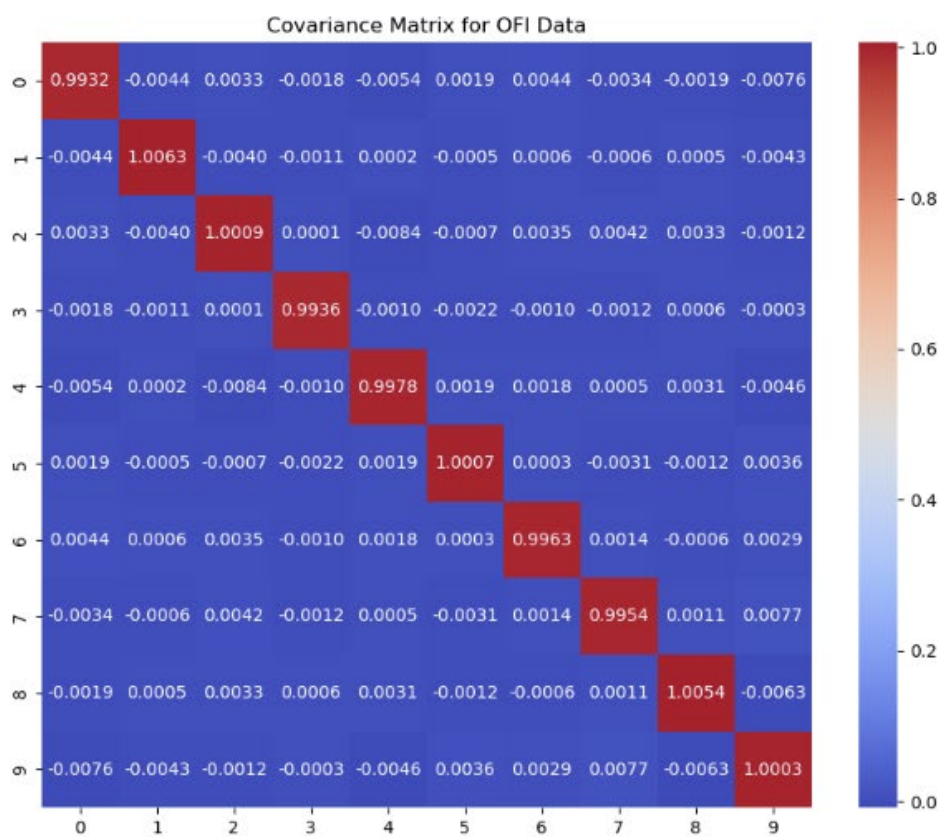
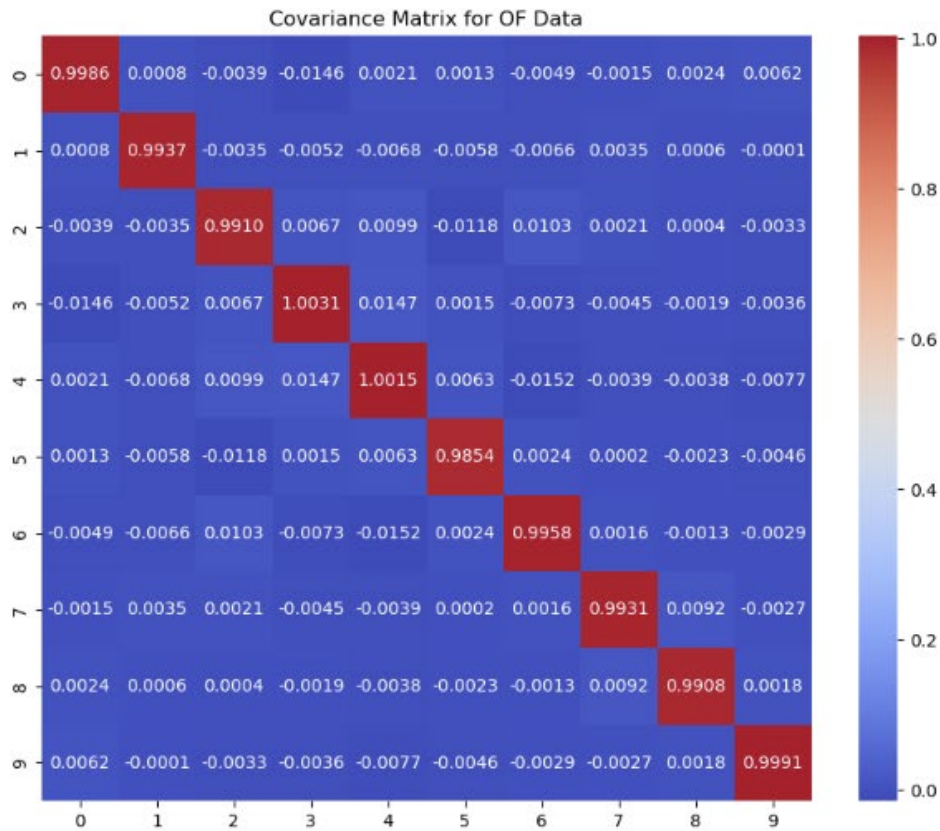
Predictions



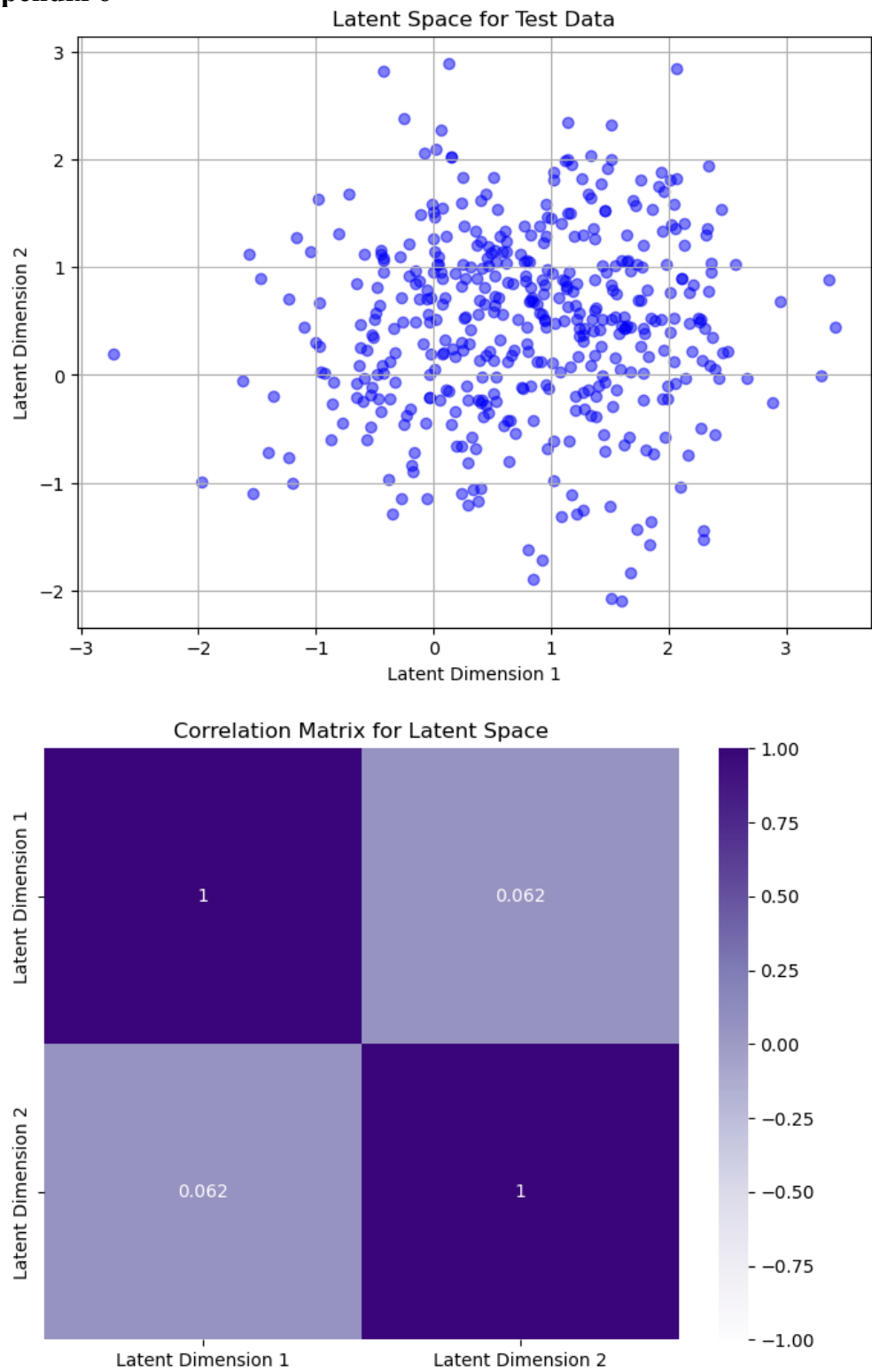
Appendix 4



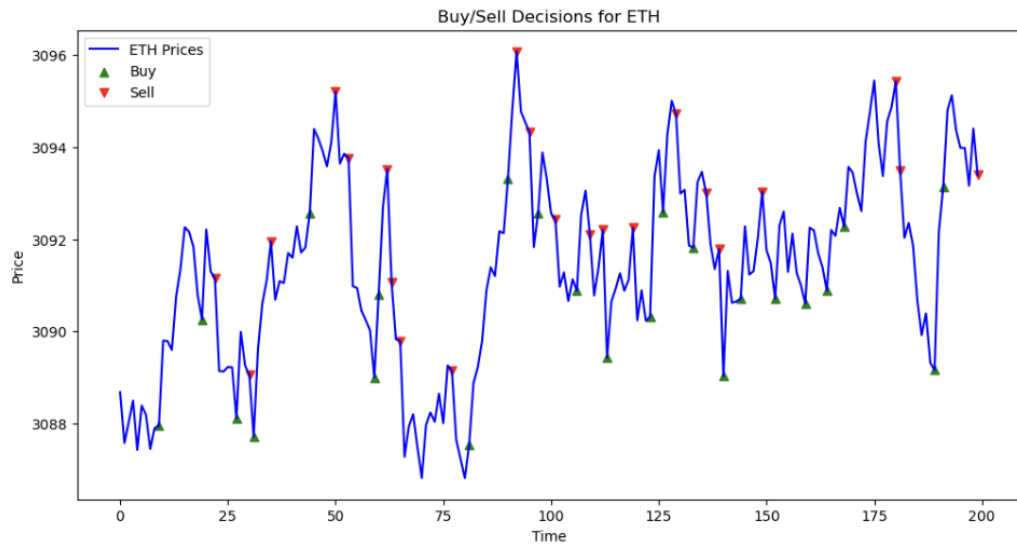
## Appendix 5



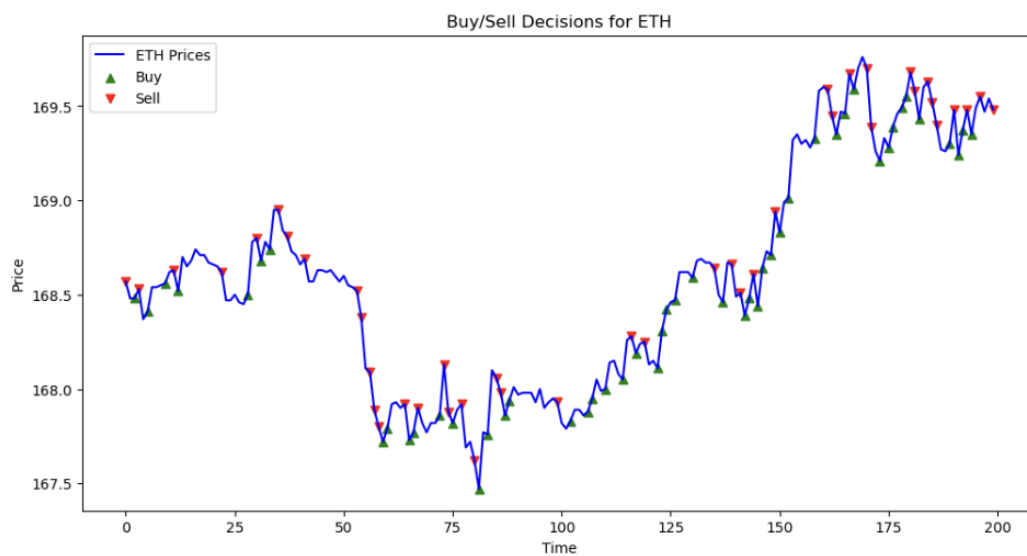
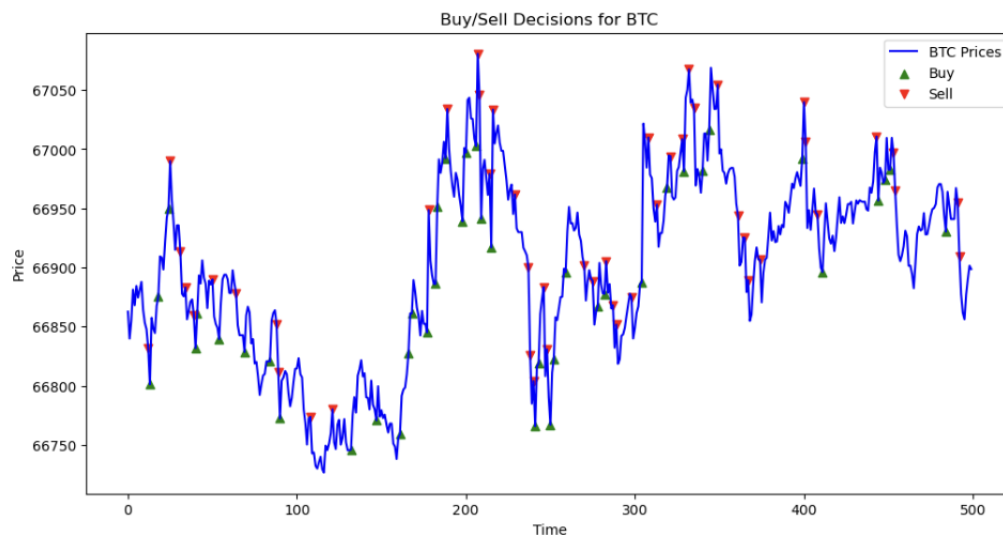
Appendix 6



## Appendix 7



Gain: 59.869999999993524



Gain: 21.390000000000327

## References

1. McNally, Sean, Jason Roche, and Simon Caton. "Predicting the Price of Bitcoin Using Machine Learning." 26th Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP), March 2018, pp. 339-343. IEEE, doi:10.1109/PDP2018.2018.00060.
2. Dixon, Matthew, Diego Klabjan, and Jin Hoon Bang. "Classification-based Financial Markets Prediction using Deep Neural Networks." *Algorithmic Finance*, vol. 6, no. 3-4, December 2017, pp. 67-77. IOS Press, doi:10.3233/AF-170176.
3. Nsarang. "Stock Price Prediction Using Historical Patterns." GitHub, <https://github.com/nsarang/big-data-stock-price-forecast.git>.
4. Koker, Thomas E., and Dimitrios Koutmos. "Cryptocurrency Trading Using Machine Learning." *Journal of Risk and Financial Management*, vol. 13, no. 178, 2020, [www.mdpi.com/journal/jrfm](http://www.mdpi.com/journal/jrfm). DOI:10.3390/jrfm13080178.
5. Jiang, Tingsong, Andy Zeng. "Financial Sentiment Analysis using FinBERT with Application in Prediction Stock Movement." arXiv, 2023, <https://arxiv.org/abs/2306.02136>
6. Zeng, Qingyun, and Tingsong Jiang. "Financial Sentiment Analysis using FinBERT with Application in Prediction Stock Movement." arXiv (2023): <https://arxiv.org/abs/2306.02136>.
7. Murray, Kate, et al. "On Forecasting Cryptocurrency Prices: A Comparison of Machine Learning, Deep Learning, and Ensembles." *Forecasting* (2023): <https://doi.org/10.3390/forecast5010010>.
8. Zhang, Zihao, Stefan Zohren, and Stephen Roberts. "DeepLOB: Deep Convolutional Neural Networks for Limit Order Books." *Journal of Latex Class Files*, vol. XX, no. XX, Jan. 2020, pp. 1-11.
9. Kolm, Petter N., Jeremy Turiel, and Nicholas Westray. "Deep Order Flow Imbalance: Extracting Alpha at Multiple Horizons from the Limit Order Book." *SSRN Electronic Journal*, 2021, SSRN: 3900141. <https://ssrn.com/abstract=3900141>.
10. Jaddu, Koti S., and Paul A. Bilokon. "Combining Deep Learning on Order Books with Reinforcement Learning for Profitable Trading." *Preprint*, Department of Computing, Imperial College London, 13 September 2023.

## Author



**Diana Gorsh,**

<https://www.linkedin.com/in/diana-gorsh/>